TruSpace

Generated by Doxygen 1.8.14

Contents

1 Class Index **1**

1.1 Class List 1

2 File Index **3**

2.1 File List 3

3 Class Documentation **5**

3.1 chirME Class Reference 5

3.1.1 Detailed Description 5

3.1.2 Constructor & Destructor Documentation 6

3.1.2.1 chirME() 6

3.1.3 Member Function Documentation 6

3.1.3.1 calculate_chirME_for_ExpVal() 6

3.1.3.2 calculate_ExpVal() 6

3.1.3.3 calculate_or_read_chirME() 7

3.1.4 Member Data Documentation 7

3.1.4.1 me_chi_ortho 7

3.1.4.2 me_chi_ortho_op_l 8

3.1.4.3 me_chi_ortho_op_r 8

3.2 diag Class Reference 8

3.2.1 Detailed Description 9

3.2.2 Member Function Documentation 9

3.2.2.1 shell_double() 9

3.2.2.2 shell_ld() 9

3.2.2.3 tqli_double() . . . 10
3.2.2.4 tqli_ld() . . . 10
3.2.2.5 tred2_double() . . . 11
3.2.2.6 tred2_ld() . . . 11
3.3 filenames Class Reference . . . 12
3.4 Hamiltonian Class Reference . . . 12
3.5 NRG Class Reference . . . 13
3.5.1 Member Function Documentation . . . 13
3.5.1.1 run() . . . 13
3.6 parameters Class Reference . . . 14
3.6.1 Detailed Description . . . 16
3.6.2 Constructor & Destructor Documentation . . . 16
3.6.2.1 parameters() . . . 16
3.6.3 Member Function Documentation . . . 17
3.6.3.1 print() . . . 17
3.6.4 Member Data Documentation . . . 17
3.6.4.1 allMomSubspace . . . 17
3.6.4.2 C . . . 17
3.6.4.3 cpt . . . 17
3.6.4.4 cptsmall . . . 18
3.6.4.5 finalsubspace . . . 18
3.6.4.6 intMatrix . . . 18
3.6.4.7 maxEigen . . . 18
3.6.4.8 maxp . . . 18
3.6.4.9 Nsize . . . 18
3.6.4.10 Nstep . . . 19
3.6.4.11 nSweep . . . 19
3.6.4.12 subdim . . . 19
3.6.4.13 vermadim . . . 19
3.6.4.14 vermadimcomul . . . 19

3.7 states Class Reference . . . 19
3.7.1 Detailed Description . . . 21
3.7.2 Constructor & Destructor Documentation . . . 21
3.7.2.1 states() . . . 21
3.7.3 Member Function Documentation . . . 21
3.7.3.1 calculate_chiral() . . . 22
3.7.3.2 calculate_nonchiral() . . . 22
3.7.3.3 read_chiral() . . . 22
3.7.4 Member Data Documentation . . . 23
3.7.4.1 cstate_lev . . . 23
3.7.4.2 cstate_mode . . . 23
3.7.4.3 cstate_nmode . . . 23
3.7.4.4 cstate_ortho_lev . . . 23
3.7.4.5 cstate_ortho_mode . . . 23
3.7.4.6 cstate_ortho_nmode . . . 24
3.7.4.7 MaxChiralStates . . . 24
3.7.4.8 MaxChiralVerma . . . 24
3.7.4.9 MaxStates . . . 24
3.7.4.10 nstates_lev . . . 24
3.7.4.11 nstates_lev_each . . . 24
3.7.4.12 state_left_lev . . . 25
3.7.4.13 state_nonchiral_lev . . . 25
3.7.4.14 state_right_lev . . . 25
3.8 tensor2D< T > Class Template Reference . . . 25
3.8.1 Detailed Description . . . 26
3.8.2 Constructor & Destructor Documentation . . . 26
3.8.2.1 tensor2D() [1/2] . . . 26
3.8.2.2 tensor2D() [2/2] . . . 26
3.8.3 Member Function Documentation . . . 27
3.8.3.1 operator()() [1/2] . . . 27

3.8.3.2 operator()() [2/2] . . . 27
3.8.3.3 operator=() . . . 27
3.9 tensor3D< T > Class Template Reference . . . 28
3.9.1 Detailed Description . . . 29
3.9.2 Constructor & Destructor Documentation . . . 29
3.9.2.1 tensor3D() . . . 29
3.9.3 Member Function Documentation . . . 29
3.9.3.1 operator()() [1/2] . . . 30
3.9.3.2 operator()() [2/2] . . . 30
3.9.3.3 operator=() . . . 30
3.10 tensor4D< T > Class Template Reference . . . 31
3.10.1 Detailed Description . . . 31
3.10.2 Constructor & Destructor Documentation . . . 32
3.10.2.1 tensor4D() . . . 32
3.10.3 Member Function Documentation . . . 32
3.10.3.1 operator()() [1/2] . . . 32
3.10.3.2 operator()() [2/2] . . . 33
3.10.3.3 operator=() . . . 33
3.11 tensor5D< T > Class Template Reference . . . 33
3.11.1 Detailed Description . . . 34
3.11.2 Constructor & Destructor Documentation . . . 34
3.11.2.1 tensor5D() . . . 35
3.11.3 Member Function Documentation . . . 35
3.11.3.1 operator()() [1/2] . . . 35
3.11.3.2 operator()() [2/2] . . . 36
3.11.3.3 operator=() . . . 36
3.12 tensor6D< T > Class Template Reference . . . 36
3.12.1 Detailed Description . . . 37
3.12.2 Constructor & Destructor Documentation . . . 38
3.12.2.1 tensor6D() . . . 38
3.12.3 Member Function Documentation . . . 38
3.12.3.1 operator()() [1/2] . . . 38
3.12.3.2 operator()() [2/2] . . . 39
3.12.3.3 operator=() . . . 39
3.13 worker Class Reference . . . 40
3.13.1 Detailed Description . . . 40

4 File Documentation **41**

4.1 block.h File Reference . . . 41

4.1.1 Detailed Description . . . 41

4.1.2 Function Documentation . . . 41

4.1.2.1 blockrec() . . . 42

4.2 characters.h File Reference . . . 43

4.2.1 Detailed Description . . . 43

4.2.2 Function Documentation . . . 44

4.2.2.1 characexpan() . . . 44

4.2.2.2 chisymb() . . . 44

4.2.2.3 determine_partition() . . . 45

4.2.2.4 invphi() . . . 45

4.2.2.5 ksymb1() . . . 46

4.2.2.6 ksymb2() . . . 46

4.2.2.7 partitions() . . . 46

4.3 chirME.h File Reference . . . 47

4.3.1 Detailed Description . . . 47

4.4 config.h File Reference . . . 47

4.4.1 Detailed Description . . . 48

4.4.2 Macro Definition Documentation . . . 48

4.4.2.1 MaxNumGrade . . . 48

4.4.2.2 MaxPart . . . 48

4.5 cpt.h File Reference . . . 48

4.5.1 Detailed Description . . . 49

4.5.2 Function Documentation . . . 49

4.5.2.1 dot() . . . 49

4.6 diag.h File Reference . . . 50

4.6.1 Detailed Description . . . 50

4.7 element_computation.h File Reference . . . 50

4.7.1 Detailed Description . . . 51

4.7.2 Function Documentation . . . 51
4.7.2.1 element_computation() . . . 51
4.7.2.2 fac() . . . 52
4.8 gamma.h File Reference . . . 52
4.8.1 Detailed Description . . . 52
4.8.2 Function Documentation . . . 52
4.8.2.1 gamma() . . . 53
4.9 parameters.h File Reference . . . 53
4.9.1 Detailed Description . . . 53
4.10 states.h File Reference . . . 54
4.10.1 Detailed Description . . . 54
4.11 tensors.h File Reference . . . 54
4.11.1 Detailed Description . . . 55
4.12 truspace.cpp File Reference . . . 55
4.12.1 Detailed Description . . . 55
4.13 worker.h File Reference . . . 56
4.13.1 Detailed Description . . . 56

Index **57**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

chirME
: Class that contains the chiral matrix elements of perturbing operators and of the operator to calculate expectation values . . . 5

diag
: Class containg a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double . . . 8

filenames . . . 12

Hamiltonian . . . 12

NRG . . . 13

parameters
: Input parameter class . . . 14

states
: Class that contains and calculates chiral states by removing null vectors from each verma module 19

tensor2D< T >
: Template class for 2D tensors This classes do not provide an allocator . . . 25

tensor3D< T >
: Template class for 3D tensors This classes do not provide an allocator . . . 28

tensor4D< T >
: Template class for 4D tensors This classes do not provide an allocator . . . 31

tensor5D< T >
: Template class for 5D tensors This classes do not provide an allocator . . . 33

tensor6D< T >
: Template class for 6D tensors This classes do not provide an allocator . . . 36

worker
: Driver Class for states, matrix element and NRG calculation . . . 40

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

block.h
: Helper function identify symmetry subspace by the analysis of the interaction matrix 41

characters.h
: Set of helper functions to calculate integers partitions and manage GiNaC computation of character expansion 43

chirME.h
: Class that contains the chiral matrix elements of perturbing operators and operator to calculate expectation values 47

config.h
: Configuration file holding version number, some hard-coded limits that effect array dimensions, constants and macros 47

cpt.h
: Set of helper function to calculate structure constant of Minimal CFTs 48

diag.h
: Contains the declaration of the diag Class, which provides a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double 50

element_computation.h
: Routines to calculate matrix element and overlap of conformal states 50

filenames.h . . . . . **??**

gamma.h
: Computation of gamma function 52

Hamiltonian.h . . . . . **??**

NRG.h . . . . . **??**

parameters.h
: Input parameter class 53

states.h
: Class that contains and calculates chiral states by removing null vectors from each verma module 54

tensors.h
: Set of helper templates class to manage 2D, 3D, 4D, 5D, 6D tensors 54

truspace.cpp
: Main program of Truspace (driver routine) 55

worker.h
: Contains driver Class for states, matrix element and NRG calculation 56

Chapter 3

Class Documentation

3.1 chirME Class Reference

Class that contains the chiral matrix elements of perturbing operators and of the operator to calculate expectation values.

`#include <chirME.h>`

Public Member Functions

- chirME (parameters &p, states &st, int subspace)

  Constructor.
- ~chirME ()

  Destructor.
- void calculate_or_read_chirME (parameters &p, filenames &fnames, states &st, int subspace)

  Routine that either calculates or reads from file chiral matrix elemment.
- void calculate_chirME_for_ExpVal (parameters &p, states &st, int subspace)

  Routine that calculates chiral matrix element for the operator whose expectation values are asked.
- void calculate_ExpVal (parameters &p, states &st, double *coeff, int size, int Momentum, std::string fname, double R)

  Routine that calculates the expectation value on the eigenvectors of the perturbed system of the given operator.

Public Attributes

- tensor5D< long double > me_chi_ortho

  Contains the chiral matrix element for each perturbing operator.
- tensor4D< long double > me_chi_ortho_op_l

  Contains the chiral matrix element for the chiral component of the operator whose expecation values are asked.
- tensor4D< long double > me_chi_ortho_op_r

  Contains the chiral matrix element for the anti-chiral component of the operator whose expecation values are asked.

3.1.1 Detailed Description

Class that contains the chiral matrix elements of perturbing operators and of the operator to calculate expectation values.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 chirME()

```
chirME::chirME (
            parameters & p,
            states & st,
            int subspace )
```

Constructor.

Parameters

in	*p*	Reference to a paramters object
in	*st*	Reference to a states object
in	*subspace*	Index of the subspace to work on

3.1.3 Member Function Documentation

3.1.3.1 calculate_chirME_for_ExpVal()

```
void chirME::calculate_chirME_for_ExpVal (
            parameters & p,
            states & st,
            int subspace )
```

Routine that calculates chiral matrix element for the operator whose expectation values are asked.

Parameters

in	*p*	Reference to a paramters object
in	*st*	Reference to a states object
in	*subspace*	Index of the subspace to work on

3.1.3.2 calculate_ExpVal()

```
void chirME::calculate_ExpVal (
            parameters & p,
            states & st,
            double * coeff,
```

```
            int size,
            int Momentum,
            std::string fname,
            double R )
```

Routine that calculates the expectation value on the eigenvectors of the perturbed system of the given operator.

Parameters

in	*p*	Reference to a paramters object
in	*st*	Reference to a states object
in	*coeff*	Matrix containing the eigenvectors of the perturbed system in rows
in	*size*	Number of components of each eigenvectors (size of the computational basis)
in	*Momentum*	Momentum subspace
in	*fname*	Output file name
in	*R*	value of the radius R

3.1.3.3 calculate_or_read_chirME()

```
void chirME::calculate_or_read_chirME (
            parameters & p,
            filenames & fnames,
            states & st,
            int subspace )
```

Routine that either calculates or reads from file chiral matrix elemment.

Parameters

in	*p*	Reference to a paramters object
in	*fnames*	Reference to a filenames object
in	*st*	Reference to a states object
in	*subspace*	Index of the subspace to work on

3.1.4 Member Data Documentation

3.1.4.1 me_chi_ortho

```
tensor5D<long double> chirME::me_chi_ortho
```

Contains the chiral matrix element for each perturbing operator.

Indexed as (perturbation_number, left Verma mod, right Verma mod, left states number, right states number)

3.1.4.2 me_chi_ortho_op_l

```
tensor4D<long double> chirME::me_chi_ortho_op_l
```

Contains the chiral matrix element for the chiral component of the operator whose expecation values are asked.

Indexed as (left Verma mod, right Verma mod, left state number, right state number)

3.1.4.3 me_chi_ortho_op_r

```
tensor4D<long double> chirME::me_chi_ortho_op_r
```

Contains the chiral matrix element for the anti-chiral component of the operator whose expecation values are asked.

Indexed as (left Verma mod, right Verma mod, left state number, right state number)

The documentation for this class was generated from the following files:

- chirME.h
- chirME.cpp

3.2 diag Class Reference

Class containg a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double.

```
#include <diag.h>
```

Public Member Functions

- void tred2_ld (tensor2D< long double > &a, int n, long double *d, long double *e, int ev_flag)

  Routine reduces a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.
- void tqli_ld (long double *d, long double *e, int n, tensor2D< long double > &z, int ev_flag)

  Routine educes a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.
- void shell_ld (unsigned long n, long double d[ ], long double *r[ ])

  Routine to sort an array in place, also providing sorting indices.
- void tred2_double (double *a[ ], int n, double d[ ], double e[ ], int ev_flag)

  Routine reduces a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.
- void tqli_double (double d[ ], double e[ ], int n, double *z[ ], int ev_flag)

  Routine educes a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.
- void shell_double (unsigned long n, double d[ ], double *r[ ])

  Routine to sort an array in place, also providing sorting indices.

3.2.1 Detailed Description

Class containg a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double.

3.2.2 Member Function Documentation

3.2.2.1 shell_double()

```
void diag::shell_double (
            unsigned long n,
            double d[],
            double * r[] )
```

Routine to sort an array in place, also providing sorting indices.

Long double version.

Parameters

in	*n*	The size of the array to sort
	[in/out]	d The array to sort. On output, it contains the sorted array
out	*r*	Matrix of size n x n+1 contaning the sorting indeces

3.2.2.2 shell_ld()

```
void diag::shell_ld (
            unsigned long n,
            long double d[],
            long double * r[] )
```

Routine to sort an array in place, also providing sorting indices.

Class containg a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double.

Long double version.

Parameters

in	*n*	The size of the array to sort
	[in/out]	d The array to sort. On output, it contains the sorted array
out	*r*	Matrix of size n x n+1 contaning the sorting indeces

Author

Robert Konik - BNL, Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

November 2016

3.2.2.3 tqli_double()

```
void diag::tqli_double (
            double d[],
            double e[],
            int n,
            double * z[],
            int ev_flag )
```

Routine educes a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.

Double version.

Parameters

	[in/out]	d Array of size n that contains the diagonal element of the tridiagonal matrix. On output, it contains the eigenvalues.
in	*e*	Array of size n that contains the subdiagonal elements of the tridiagonal matrix in its last n-1 positions
in	*n*	Size of the matrix
out	*e*	Reference of tensor2D<long double > of size n x n. It will contain the eigenvectors is ev_flag is set to 1.
in	*ev_flag*	Set to 1 if eigenvectors are required, zero otherwise

3.2.2.4 tqli_ld()

```
void diag::tqli_ld (
            long double * d,
            long double * e,
            int n,
            tensor2D< long double > & z,
            int ev_flag )
```

Routine educes a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.

Double version.

Parameters

	[in/out]	d Array of size n that contains the diagonal element of the tridiagonal matrix. On output, it contains the eigenvalues.
in	*e*	Array of size n that contains the subdiagonal elements of the tridiagonal matrix in its last n-1 positions
in	*n*	Size of the matrix
out	*e*	Reference of tensor2D<long double > of size n x n. It will contain the eigenvectors is ev_flag is set to 1.
in	*ev_flag*	Set to 1 if eigenvectors are required, zero otherwise

3.2.2.5 tred2_double()

```
void diag::tred2_double (
            double * a[],
            int n,
            double d[],
            double e[],
            int ev_flag )
```

Routine reduces a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.

Long double version.

Parameters

in	*a*	Reference of tensor2D<long double > of size n x n. The matrix to be reduced
in	*n*	Size of the matrix
out	*d*	Array of size n that will contain the diagonal element of the tridiagonal matrix
out	*e*	Array of size n that will contain the subdiagonal elements of the tridiagonal matrix in its last n-1 positions
in	*ev_flag*	Set to 1 if eigenvectors are required, zero otherwise

3.2.2.6 tred2_ld()

```
void diag::tred2_ld (
            tensor2D< long double > & a,
            int n,
            long double * d,
            long double * e,
            int ev_flag )
```

Routine reduces a real symmetric matrix to a symmetric tridiagonal matrix using and accumulating orthogonal similarity transformations.

Long double version.

Parameters

in	*a*	Reference of tensor2D<long double > of size n x n. The matrix to be reduced
in	*n*	Size of the matrix
out	*d*	Array of size n that will contain the diagonal element of the tridiagonal matrix
out	*e*	Array of size n that will contain the subdiagonal elements of the tridiagonal matrix in its last n-1 positions
in	*ev_flag*	Set to 1 if eigenvectors are required, zero otherwise

The documentation for this class was generated from the following files:

- diag.h
- diag.cpp

3.3 filenames Class Reference

Public Member Functions

- **filenames** (parameters &, int)

Public Attributes

- std::string **EigenName**
- std::string **EigenNameFinal**
- std::string **EigenVectName**
- std::string **MeOpName**
- std::string **StoreName**
- std::string **OverlapName**
- std::string **StateName**
- std::string * **MeChirName**

The documentation for this class was generated from the following files:

- filenames.h
- filenames.cpp

3.4 Hamiltonian Class Reference

Public Member Functions

- **Hamiltonian** (parameters &, states &, chirME &, int)
- void **init** ()
- void **print_eigenValues** (int, int, double *, std::string, int)
- void **print_eigenVectors** (double *, std::string, int)
- void **NRG_block_A** (double *)
- void **NRG_block_BC** (double *, int, double *)
- void **NRG_block_D** (int)
- void **sweep_block_BC** (double *, int, double *, double *, int, int)

Public Attributes

- tensor2D< double > **H**
- int **Msize**
- double **R**
- int **Momentum**
- parameters & **p**
- states & **st**
- chirME & **me**
- int **subspace**

The documentation for this class was generated from the following files:

- Hamiltonian.h
- Hamiltonian.cpp

3.5 NRG Class Reference

Public Member Functions

- void run (parameters &, Hamiltonian &, filenames &)

Public Attributes

- tensor2D< double > **EigenVectors**
- int **NIters**
- int **NSweeps**

3.5.1 Member Function Documentation

3.5.1.1 run()

```
void NRG::run (
            parameters & p,
            Hamiltonian & H,
            filenames & fnames )
```

diagonalize Hamiltonian and sort for relevant states

The documentation for this class was generated from the following files:

- NRG.h
- NRG.cpp

3.6 parameters Class Reference

Input parameter class.

`#include <parameters.h>`

Public Member Functions

- parameters (const char *)
  Constructor.
- void print () const
  Prints the input paramters, Verma modules size, Hilbert space size, subspace analysis and structure constant.
- void analyse_subspaces ()
  Calculates Verma modules size, Hilbert space size, subspace analysis and structure constant.
- ~parameters ()
  Destructor.

Public Attributes

- int p
  p index in Kac matrix
- int q
  q index in Kac matrix
- int numPert
  number of pertubations
- int maxLev
  Maximum conformal level to include.
- int minLev
  Minimum conformal level to include (usually 0)
- int maxMomentum
  Maximum momentum considered.
- bool allMomSubspace
  True to diagonalize all momentum subspaces together.
- int maxEigen
  Maximum number of eigenvalues printed in the output file.
- int * r
  array containing Kac r index of perturbations
- int * s
  array containing Kac s index of perturbations
- double * deltas
  array containing chiral conformal dimensions of each perturbation
- double * pConst
  array containing the constant for each perturbation
- int rSteps
  number of values of the radius to be considered
- double rInc
  increment of radius at each step
- double rStart
  initial value of the radius

- int Nsize

  basic sizes the NRG matrix.
- int Nstep

  number of states added at each NRG iteration.
- bool TCSA

  True for diagonalization, false to have just Kac table structure constants and subspace analysis.
- bool NRG

  True for NRG, false for full diagonalization.
- int nSweep

  number of NRG sweep.
- bool SAVE_STATES

  True to save computationale basis to file, false not to save them.
- bool READ_STATES

  True to read computationale basis from file, false to calculate them.
- bool SAVE_ME

  True to save chiral matrix element of perturbing fields to file, false not to save them.
- bool READ_ME

  True to read chiral matrix element of perturbing fields to file, false to calculate them.
- bool calcExpVal

  True to calculate expectation values on the eigenstates of the perturbed system of a primary operator, false not to.
- int rl_op

  r index of holomorphic component of the operator to calculate expectation values
- int sl_op

  s index of holomorphic component of the operator to calculate expectation values
- int nModes_l_op

  number of virasoro operator applied to the holomorphic component of the primary operator (set 0 to evaluate operator itself)
- int * mode_l_op

  array containing the virasoro modes applied to the holomorphic component of the field
- int rr_op

  r index of anti holomorphic component of the operator to calculate expectation values
- int sr_op

  s index of anti holomorphic component of the operator to calculate expectation values
- int nModes_r_op

  number of virasoro operator applied to the anti holomorphic component of the primary operator (set 0 to evaluate operator itself)
- int * mode_r_op

  array containing the virasoro modes applied to the anti holomorphic component of the field
- bool onlyDiagExpV

  True to calculate only diagonal expectation values, false for the full matrix.
- bool saveEigenvec

  True to save eigenvectors to file, false not to save them.
- double C

  central charge.
- int maxp

  p size of Kac table.
- int maxq

  q size of Kac table Calculated by the code
- int maxl

  total size of Kac table Calculated by the code
- int numspace

number of subspace found by analysing the system Calculated by the code

- int * maxNumStates

  array containng the maximum number of states for a given level Calculated by the code

- tensor2D< double > Kac

  Contains the Kac table, meaning the weight of primary fields, indexed with standard r,s indeces.

- tensor6D< std::complex< long double > > cpt

  Structure constant of the model.

- tensor3D< std::complex< long double > > cptsmall

  Structure constant of the model, indexed with field index instead of r,s.

- tensor3D< int > vermadim

  contains the number of states for each level of each verma module.

- tensor3D< int > vermadimcomul

  contains the cumulative number of states up to each level of each verma module.

- tensor2D< int > finalsubspace

  contains the label of the verma module in each subspace.

- tensor2D< int > intMatrix

  contains the sketch of the interaction matrix, where each element is a verma module.

- int * subdim

  contains the number of vermamodule in a give symmetry subspace.

- int * leveldim

  Contains the.

- tensor2D< int > subdimStates

  contains the predicted number of predicted non-chiral states in each subspace, and at given momentum

3.6.1 Detailed Description

Input parameter class.

Big fuzzy object that contains

- input paramenter
- Kac table
- structure constant
- verma module in each subspace

3.6.2 Constructor & Destructor Documentation

3.6.2.1 parameters()

```
parameters::parameters (
            const char * input_file_name )
```

Constructor.

Parameters

in	*The*	name of the input file

3.6.3 Member Function Documentation

3.6.3.1 print()

```
void parameters::print ( ) const
```

Prints the input paramters, Verma modules size, Hilbert space size, subspace analysis and structure constant.

Needs to be called after analyse_subspaces()

3.6.4 Member Data Documentation

3.6.4.1 allMomSubspace

```
bool parameters::allMomSubspace
```

True to diagonalize all momentum subspaces together.

Useful for calculation of momentum-carrying operator matrix elements. False diagonalize momentum subspaces separately.

3.6.4.2 C

```
double parameters::C
```

central charge.

Calculated by the code

3.6.4.3 cpt

```
tensor6D< std::complex<long double> > parameters::cpt
```

Structure constant of the model.

Calculated by the code

3.6.4.4 cptsmall

```
tensor3D< std::complex<long double> > parameters::cptsmall
```

Structure constant of the model, indexed with field index instead of r,s.

Calculated by the code

3.6.4.5 finalsubspace

```
tensor2D< int > parameters::finalsubspace
```

contains the label of the verma module in each subspace.

Calculated by the code

3.6.4.6 intMatrix

```
tensor2D< int > parameters::intMatrix
```

contains the sketch of the interaction matrix, where each element is a verma module.

Calculated by the code

3.6.4.7 maxEigen

```
int parameters::maxEigen
```

Maximum number of eigenvalues printed in the output file.

3.6.4.8 maxp

```
int parameters::maxp
```

p size of Kac table.

Calculated by the code

3.6.4.9 Nsize

```
int parameters::Nsize
```

basic sizes the NRG matrix.

3.6.4.10 Nstep

```
int parameters::Nstep
```

number of states added at each NRG iteration.

The matrix diagonalized in NRG is Nsize+Nstep

3.6.4.11 nSweep

```
int parameters::nSweep
```

number of NRG sweep.

Set 0 for basic NRG

3.6.4.12 subdim

```
int* parameters::subdim
```

contains the number of vermamodule in a give symmetry subspace.

Calculated by the code

3.6.4.13 vermadim

```
tensor3D< int > parameters::vermadim
```

contains the number of states for each level of each verma module.

The first two indeces are the r,s indeces of the primary field, the third is the level

3.6.4.14 vermadimcomul

```
tensor3D< int > parameters::vermadimcomul
```

contains the cumulative number of states up to each level of each verma module.

The first two indeces are the r,s indeces of the primary field, the third is the level

The documentation for this class was generated from the following files:

- parameters.h
- parameters.cpp

3.7 states Class Reference

Class that contains and calculates chiral states by removing null vectors from each verma module.

```
#include <states.h>
```

Public Member Functions

- states (parameters &p, int subspace)

  Constructor.
- ~states ()

  Destructor.
- void calculate_chiral (parameters &p, int subspace, filenames &fname)

  Routine to calculate chiral states, stored internally in the object itself.
- void calculate_nonchiral (parameters &p, int subspace, int mom)

  Routine to calculate nonchiral states, stored internally in the object itself.
- void read_chiral (parameters &p, int subspace, filenames &fname)

  Routine to read chiral states from file, stored internally in the object itself.

Public Attributes

- int MaxChiralStates

  The maximum number of chiral states, calculated from character.
- int MaxChiralVerma

  The maximum number of chiral states in a Verma module, calculated from character.
- int MaxStates

  The maximum number of nonchiral states, calculated from character.
- int nstate

  The total number of nonchiral states in the subspace.
- double * state_en

  Array contaning the conformal energy of every nonchiral state.
- double * state_eff_en

  Array contaning the effective energy of every nonchiral state.
- int * state_left

  Array that, given the global index of nonchiral states, returns the index in the correct Verma module, for the bra state.
- int * state_right

  Array that, given the global index of nonchiral states, returns the index in the correct Verma module, for the ket state.
- int * state_vmod

  Array that, given the global index of nonchiral states, returns the index of the correct Verma module.
- long double * h

  array that contains the conformal dimension of the primary field labelling the verma module for each verma module in the subspace
- tensor3D< long double > cstate_ortho_modecoeff

  3D Tensor containing the coefficient of orthogonal conformal states on the non-orthogonal basis
- tensor3D< long double > me_chi_0

  3D Tensor containing the overlaps of conformal states, used to remove null vectors
- tensor2D< long double > norm

  2D Tensor containing norms of
- tensor3D< long double > cstate_mode

  3D Tensor that contains the strings of lowering operators that define a non-orthogonal chiral state.
- tensor2D< long double > cstate_norm

  Norm of conformal chiral states.
- int * num_chir

  Array to store the number of chiral non-null states at each level.
- tensor2D< int > cstate_lev

  Array that returns the level of a chiral non-orthogonal states, given its index.

- tensor2D< int > cstate_nmode

  Array that returns the number of modes for a chiral non-orthogonal states , given its index.
- tensor2D< int > cstate_ortho_lev

  Array that returns the level of a chiral orthogonal states, given its index.
- tensor2D< int > cstate_ortho_nmode

  Array that returns the number of modes for a chiral orthogonal states, given its index.
- tensor3D< int > cstate_ortho_mode

  3D Tensor that contains the strings of lowering operators that define an orthogonal chiral state.
- int MaxKac

  Stores the size of the Kac determinant at each level.
- tensor2D< int > nstates_lev

  Number of chiral state up to a given level.
- tensor2D< int > nstates_lev_each

  Number of chiral states in each level.
- tensor3D< int > state_right_lev

  Used only with sweep.
- tensor3D< int > state_left_lev

  Used only with sweep.
- tensor4D< int > state_nonchiral_lev

  Used only with sweep.

3.7.1 Detailed Description

Class that contains and calculates chiral states by removing null vectors from each verma module.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 states()

```
states::states (
            parameters & p,
            int subspace )
```

Constructor.

Parameters

in	*p*	Reference to a parameters object
in	*subspace*	Subspace to work on

3.7.3 Member Function Documentation

3.7.3.1 calculate_chiral()

```
void states::calculate_chiral (
            parameters & p,
            int subspace,
            filenames & fname )
```

Routine to calculate chiral states, stored internally in the object itself.

Parameters

in	*p*	Reference to a parameters object
in	*subspace*	Subspace to work on
in	*fname*	Reference to a filenames object

3.7.3.2 calculate_nonchiral()

```
void states::calculate_nonchiral (
            parameters & p,
            int subspace,
            int mom )
```

Routine to calculate nonchiral states, stored internally in the object itself.

Parameters

in	*p*	Reference to a parameters object
in	*subspace*	Subspace to work on
in	*mom*	Momentum subspace to work on

3.7.3.3 read_chiral()

```
void states::read_chiral (
            parameters & p,
            int subspace,
            filenames & fname )
```

Routine to read chiral states from file, stored internally in the object itself.

Parameters

in	*p*	Reference to a parameters object
in	*subspace*	Subspace to work on
in	*fname*	Reference to a filenames object

3.7.4 Member Data Documentation

3.7.4.1 cstate_lev

```
tensor2D< int > states::cstate_lev
```

Array that returns the level of a chiral non-orthogonal states, given its index.

The first index runs on Verma modules in the subspace, the second is the index of the chiral state

3.7.4.2 cstate_mode

```
tensor3D< long double > states::cstate_mode
```

3D Tensor that contains the strings of lowering operators that define a non-orthogonal chiral state.

The first index runs on Verma modules in the subspace, the second is the index of the chiral state. The third on the string of indeces

3.7.4.3 cstate_nmode

```
tensor2D< int > states::cstate_nmode
```

Array that returns the number of modes for a chiral non-orthogonal states , given its index.

The first index runs on Verma modules in the subspace, the second is the index of the chiral state

3.7.4.4 cstate_ortho_lev

```
tensor2D< int > states::cstate_ortho_lev
```

Array that returns the level of a chiral orthogonal states, given its index.

The first index runs on Verma modules in the subspace, the second is the index of the chiral state

3.7.4.5 cstate_ortho_mode

```
tensor3D< int > states::cstate_ortho_mode
```

3D Tensor that contains the strings of lowering operators that define an orthogonal chiral state.

The first index runs on Verma modules in the subspace, the second is the index of the chiral state. The third on the string of indeces

3.7.4.6 cstate_ortho_nmode

```
tensor2D< int > states::cstate_ortho_nmode
```

Array that returns the number of modes for a chiral orthogonal states, given its index.

The first index runs on Verma modules in the subspace, the second is the index of the chiral state

3.7.4.7 MaxChiralStates

```
int states::MaxChiralStates
```

The maximum number of chiral states, calculated from character.

Used for array allocation

3.7.4.8 MaxChiralVerma

```
int states::MaxChiralVerma
```

The maximum number of chiral states in a Verma module, calculated from character.

Used for array allocation

3.7.4.9 MaxStates

```
int states::MaxStates
```

The maximum number of nonchiral states, calculated from character.

Used for array allocation

3.7.4.10 nstates_lev

```
tensor2D< int > states::nstates_lev
```

Number of chiral state up to a given level.

The first index runs on Verma modules in the subspace, the second on levels

3.7.4.11 nstates_lev_each

```
tensor2D< int > states::nstates_lev_each
```

Number of chiral states in each level.

The first index runs on Verma modules in the subspace, the second on levels

3.7.4.12 state_left_lev

`tensor3D< int > states::state_left_lev`

Used only with sweep.

Tensor that given the verma module, the level and the index inside a level return the global index of the states of the chiral level in the verma module, for the chiral part

3.7.4.13 state_nonchiral_lev

`tensor4D< int > states::state_nonchiral_lev`

Used only with sweep.

Tensor that given the verma module, the level of a non chiral states, and the index of its chiral and antichiral components gives the index on the non chiral state in the global indexing

3.7.4.14 state_right_lev

`tensor3D< int > states::state_right_lev`

Used only with sweep.

Tensor that given the verma module, the level and the index inside a level return the global index of the states of the chiral level in the verma module, for the anti-chiral part

The documentation for this class was generated from the following files:

- states.h
- states.cpp

3.8 tensor2D< T > Class Template Reference

Template class for 2D tensors This classes do not provide an allocator.

`#include <tensors.h>`

Public Member Functions

- tensor2D ()
  Default Constructor, it allocates a 1x1 tensor.
- tensor2D (int, int)
  Constructor to allocate custom size 2D tensor.
- ~tensor2D ()
  Destructor, deallocates data member.
- tensor2D< T > & operator= (const tensor2D< T > &)
  Assignment operator.
- T & operator() (const int, const int)
  Element access operator, with bound check.
- T operator() (const int, const int) const
  Element access operator, with bound check - Constant tensor version.

Public Attributes

- int n1

  First ensor size.
- int n2

  Second size.
- T * data

  Data segment.

3.8.1 Detailed Description

template<class T>
class tensor2D< T >

Template class for 2D tensors This classes do not provide an allocator.

They are allocated automatically using the costum constructor tesor?D(int int ....) The default constructor can be used, but only to create dummy tensors, that needs to be assigned using the assignment operator (=)

Memebers are all public.

- n1, n2, ... contains the dimensions
- the pointer data contains the data in col major order, such that it can be pass to BLAS/LAPACK easily as A.data
- reading and writing single elements is done using the () operator so element 1,2,5 of tensor A is accessed A(1,2,5)

3.8.2 Constructor & Destructor Documentation

3.8.2.1 tensor2D() [1/2]

```
template<class T >
tensor2D< T >::tensor2D ( )
```

Default Constructor, it allocates a 1x1 tensor.

3.8.2.2 tensor2D() [2/2]

```
template<class T >
tensor2D< T >::tensor2D (
            int n1_in,
            int n2_in )
```

Constructor to allocate custom size 2D tensor.

Parameters

in	*n1_in*	First dimension
in	*n2_in*	Second dimension

3.8.3 Member Function Documentation

3.8.3.1 operator()() [1/2]

```
template<class T >
T & tensor2D< T >::operator() (
            const int i1,
            const int i2 )
```

Element access operator, with bound check.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access

3.8.3.2 operator()() [2/2]

```
template<class T >
T tensor2D< T >::operator() (
            const int i1,
            const int i2 ) const
```

Element access operator, with bound check - Constant tensor version.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access

3.8.3.3 operator=()

```
template<class T>
tensor2D< T > & tensor2D< T >::operator= (
            const tensor2D< T > & tens )
```

Assignment operator.

Parameters

in	*tens*	The tensor to copy return A reference to the current object, to allow expressions such as a=b=c to work

The documentation for this class was generated from the following file:

- tensors.h

3.9 tensor3D< T > Class Template Reference

Template class for 3D tensors This classes do not provide an allocator.

`#include <tensors.h>`

Public Member Functions

- tensor3D ()

  Default Constructor, it allocates a 1x1x1 tensor.
- tensor3D (int, int, int)

  Constructor to allocate custom size 3D tensor.
- ~tensor3D ()

  Destructor, deallocates data member.
- tensor3D< T > & operator= (const tensor3D< T > &)

  Assignment operator.
- T & operator() (const int, const int, const int)

  Element access operator, with bound check.
- T operator() (const int, const int, const int) const

  Element access operator, with bound check - Constant tensor version.

Public Attributes

- int n1

  First tensor size.
- int n2

  Second tensor size.
- int n3

  Third tensor size.
- T * data

  Data segment.

3.9.1 Detailed Description

template<class T>
class tensor3D< T >

Template class for 3D tensors This classes do not provide an allocator.

They are allocated automatically using the costum constructor tesor?D(int int ....) The default constructor can be used, but only to create dummy tensors, that needs to be assigned using the assignment operator (=)

Memebers are all public.

- n1, n2, ... contains the dimensions
- the pointer data contains the data in col major order, such that it can be pass to BLAS/LAPACK easily as A.data
- reading and writing single elements is done using the () operator so element 1,2,5 of tensor A is accessed A(1,2,5)

3.9.2 Constructor & Destructor Documentation

3.9.2.1 tensor3D()

```
template<class T >
tensor3D< T >::tensor3D (
            int n1_in,
            int n2_in,
            int n3_in )
```

Constructor to allocate custom size 3D tensor.

Parameters

in	*n1_in*	First dimension
in	*n2_in*	Second dimension
in	*n3_in*	Thrid dimension

3.9.3 Member Function Documentation

3.9.3.1 operator()() [1/2]

```
template<class T >
T & tensor3D< T >::operator() (
            const int i1,
            const int i2,
            const int i3 )
```

Element access operator, with bound check.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access

3.9.3.2 operator()() [2/2]

```
template<class T >
T tensor3D< T >::operator() (
            const int i1,
            const int i2,
            const int i3 ) const
```

Element access operator, with bound check - Constant tensor version.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access

3.9.3.3 operator=()

```
template<class T>
tensor3D< T > & tensor3D< T >::operator= (
            const tensor3D< T > & tens )
```

Assignment operator.

Parameters

in	*tens*	The tensor to copy return A reference to the current object, to allow expressions such as a=b=c to work

The documentation for this class was generated from the following file:

- tensors.h

3.10 tensor4D< T > Class Template Reference

Template class for 4D tensors This classes do not provide an allocator.

`#include <tensors.h>`

Public Member Functions

- tensor4D ()
  Default Constructor, it allocates a 1x1x1x1 tensor.
- tensor4D (int, int, int, int)
  Constructor to allocate custom size 4D tensor.
- ~tensor4D ()
  Destructor, deallocates data member.
- tensor4D< T > & operator= (const tensor4D< T > &)
  Assignment operator.
- T & operator() (const int, const int, const int, const int)
  Element access operator, with bound check.
- T operator() (const int, const int, const int, const int) const
  Element access operator, with bound check - Constant tensor version.

Public Attributes

- int n1
  First tensor size.
- int n2
  Second tensor size.
- int n3
  Third tensor size.
- int n4
  Fourth tensor size.
- T * data
  Data segment.

3.10.1 Detailed Description

template<class T>
class tensor4D< T >

Template class for 4D tensors This classes do not provide an allocator.

They are allocated automatically using the costum constructor tesor?D(int int ....) The default constructor can be used, but only to create dummy tensors, that needs to be assigned using the assignment operator (=)

Memebers are all public.

- n1, n2, ... contains the dimensions
- the pointer data contains the data in col major order, such that it can be pass to BLAS/LAPACK easily as A.data
- reading and writing single elements is done using the () operator so element 1,2,5 of tensor A is accessed A(1,2,5)

3.10.2 Constructor & Destructor Documentation

3.10.2.1 tensor4D()

```
template<class T >
tensor4D< T >::tensor4D (
            int n1_in,
            int n2_in,
            int n3_in,
            int n4_in )
```

Constructor to allocate custom size 4D tensor.

Parameters

in	*n1_in*	First dimension
in	*n2_in*	Second dimension
in	*n3_in*	Third dimension
in	*n4_in*	Fourth dimension

3.10.3 Member Function Documentation

3.10.3.1 operator()() [1/2]

```
template<class T >
T & tensor4D< T >::operator() (
            const int i1,
            const int i2,
            const int i3,
            const int i4 )
```

Element access operator, with bound check.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access
in	*i4*	Fourth index of the element to access

3.10.3.2 operator()() [2/2]

```
template<class T >
T tensor4D< T >::operator() (
            const int i1,
            const int i2,
            const int i3,
            const int i4 ) const
```

Element access operator, with bound check - Constant tensor version.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access
in	*i4*	Fourth index of the element to access

3.10.3.3 operator=()

```
template<class T>
tensor4D< T > & tensor4D< T >::operator= (
            const tensor4D< T > & tens )
```

Assignment operator.

Parameters

in	*tens*	The tensor to copy return A reference to the current object, to allow expressions such as a=b=c to work

The documentation for this class was generated from the following file:

- tensors.h

3.11 tensor5D< T > Class Template Reference

Template class for 5D tensors This classes do not provide an allocator.

```
#include <tensors.h>
```

Public Member Functions

- tensor5D ()

  Default Constructor, it allocates a 1x1x1x1x1 tensor.
- tensor5D (int, int, int, int, int)

Constructor to allocate custom size 5D tensor.

- ~tensor5D ()

  Destructor, deallocates data member.

- tensor5D< T > & operator= (const tensor5D< T > &)

  Assignment operator.

- T & operator() (const int, const int, const int, const int, const int)

  Element access operator, with bound check.

- T operator() (const int, const int, const int, const int, const int) const

  Element access operator, with bound check - Constant tensor version.

Public Attributes

- int n1

  First tensor size.

- int n2

  Second tensor size.

- int n3

  Third tensor size.

- int n4

  Fourth tensor size.

- int n5

  Fifth tensor size.

- T * data

  Data segment.

3.11.1 Detailed Description

template<class T>
class tensor5D< T >

Template class for 5D tensors This classes do not provide an allocator.

They are allocated automatically using the costum constructor tesor?D(int int ....) The default constructor can be used, but only to create dummy tensors, that needs to be assigned using the assignment operator (=)

Memebers are all public.

- n1, n2, ... contains the dimensions
- the pointer data contains the data in col major order, such that it can be pass to BLAS/LAPACK easily as A.data
- reading and writing single elements is done using the () operator so element 1,2,5 of tensor A is accessed A(1,2,5)

3.11.2 Constructor & Destructor Documentation

3.11.2.1 tensor5D()

```
template<class T >
tensor5D< T >::tensor5D (
            int n1_in,
            int n2_in,
            int n3_in,
            int n4_in,
            int n5_in )
```

Constructor to allocate custom size 5D tensor.

Parameters

in	*n1_in*	First dimension
in	*n2_in*	Second dimension
in	*n3_in*	Third dimension
in	*n4_in*	Fourth dimension
in	*n5_in*	Fifth dimension

3.11.3 Member Function Documentation

3.11.3.1 operator()() [1/2]

```
template<class T >
T & tensor5D< T >::operator() (
            const int i1,
            const int i2,
            const int i3,
            const int i4,
            const int i5 )
```

Element access operator, with bound check.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access
in	*i4*	Fourth index of the element to access
in	*i5*	Fifth index of the element to access

3.11.3.2 operator()() [2/2]

```
template<class T >
T tensor5D< T >::operator() (
             const int i1,
             const int i2,
             const int i3,
             const int i4,
             const int i5 ) const
```

Element access operator, with bound check - Constant tensor version.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access
in	*i4*	Fourth index of the element to access
in	*i5*	Fifth index of the element to access

3.11.3.3 operator=()

```
template<class T>
tensor5D< T > & tensor5D< T >::operator= (
             const tensor5D< T > & tens )
```

Assignment operator.

Parameters

in	*tens*	The tensor to copy return A reference to the current object, to allow expressions such as a=b=c to work

The documentation for this class was generated from the following file:

- tensors.h

3.12 tensor6D< T > Class Template Reference

Template class for 6D tensors This classes do not provide an allocator.

```
#include <tensors.h>
```

Public Member Functions

- tensor6D ()
  Default Constructor, it allocates a 1x1x1x1x1x1 tensor.
- tensor6D (int, int, int, int, int, int)
  Constructor to allocate custom size 2D tensor.
- ~tensor6D ()
  Destructor, deallocates data member.
- tensor6D< T > & operator= (const tensor6D< T > &)
  Assignment operator.
- T & operator() (const int, const int, const int, const int, const int, const int)
  Element access operator, with bound check.
- T operator() (const int, const int, const int, const int, const int, const int) const
  Element access operator, with bound check - Constant tensor version.

Public Attributes

- int n1
  First tensor size.
- int n2
  Second tensor size.
- int n3
  Third tensor size.
- int n4
  Fourth tensor size.
- int n5
  Fifth tensor size.
- int n6
  Sixth tensor size.
- T * data
  Data segment.

3.12.1 Detailed Description

template<class T>
class tensor6D< T >

Template class for 6D tensors This classes do not provide an allocator.

They are allocated automatically using the costum constructor tesor?D(int int ....) The default constructor can be used, but only to create dummy tensors, that needs to be assigned using the assignment operator (=)

Memebers are all public.

- n1, n2, ... contains the dimensions
- the pointer data contains the data in col major order, such that it can be pass to BLAS/LAPACK easily as A.data
- reading and writing single elements is done using the () operator so element 1,2,5 of tensor A is accessed A(1,2,5)

3.12.2 Constructor & Destructor Documentation

3.12.2.1 tensor6D()

```
template<class T >
tensor6D< T >::tensor6D (
            int n1_in,
            int n2_in,
            int n3_in,
            int n4_in,
            int n5_in,
            int n6_in )
```

Constructor to allocate custom size 2D tensor.

Parameters

in	*n1_in*	First dimension
in	*n2_in*	Second dimension
in	*n3_in*	Third dimension
in	*n4_in*	Fourth dimension
in	*n5_in*	Fifth dimension
in	*n6_in*	Sixth dimension

3.12.3 Member Function Documentation

3.12.3.1 operator()() [1/2]

```
template<class T >
T & tensor6D< T >::operator() (
            const int i1,
            const int i2,
            const int i3,
            const int i4,
            const int i5,
            const int i6 )
```

Element access operator, with bound check.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access
in	*i4*	Fourth index of the element to access
in	*i5*	Fifth index of the element to access
in	*i6*	Sixth index of the element to access

3.12.3.2 operator()() [2/2]

```
template<class T >
T tensor6D< T >::operator() (
            const int i1,
            const int i2,
            const int i3,
            const int i4,
            const int i5,
            const int i6 ) const
```

Element access operator, with bound check - Constant tensor version.

Parameters

in	*i1*	First index of the element to access
in	*i2*	Second index of the element to access
in	*i3*	Third index of the element to access
in	*i4*	Fourth index of the element to access
in	*i5*	Fifth index of the element to access
in	*i6*	Sixth index of the element to access

3.12.3.3 operator=()

```
template<class T>
tensor6D< T > & tensor6D< T >::operator= (
            const tensor6D< T > & tens )
```

Assignment operator.

Parameters

in	*tens*	The tensor to copy return A reference to the current object, to allow expressions such as a=b=c to work

The documentation for this class was generated from the following file:

- tensors.h

3.13 worker Class Reference

Driver Class for states, matrix element and NRG calculation.

`#include <worker.h>`

Public Member Functions

- worker ()
  Default constructor.
- ~worker ()
  Default destructor.
- void run (parameters &p)
  [in] p paramerts object

Public Attributes

- std::string EigenName
  File name of eigenvalues file.
- std::string EigenVectName
  File name of eigenvectors file.
- std::string MeOpName
  File name of expectation value file.
- std::string OverlapName
  File name of overlap file (Used when storing states or chiral matrix elements)
- std::string StateName
  File name of states file (Used when storing states)
- std::string MeChirName
  File name of chiral matrix element file (Used when storing chiral matrix elements)

3.13.1 Detailed Description

Driver Class for states, matrix element and NRG calculation.

It takes care of

- looping through the subspace found by parameters::analyze_subspace
- finding chiral and non-chiral base states in each subspace
- calling NRG routine on each subspace
- calculating Matrix element on the eigenstates found, in each subspace. If the operator connects a set of subspaces, then those subspaces are diagonalized at the same time

Author

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

November 2017

The documentation for this class was generated from the following files:

- worker.h
- worker.cpp

Chapter 4

File Documentation

4.1 block.h File Reference

Helper function identify symmetry subspace by the analysis of the interaction matrix.

```
#include "tensors.h"
```

Functions

- void blockrec (int maxl, tensor2D< complex< long double > > &a, tensor2D< int > &finalsubspace, int *subdim, int &numspace)

  Routine to identify symmetry subspace by the analysis of the interaction matrix.

4.1.1 Detailed Description

Helper function identify symmetry subspace by the analysis of the interaction matrix.

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

Copyright

GNU Public License.

4.1.2 Function Documentation

4.1.2.1 blockrec()

```
void blockrec (
            int maxl,
            tensor2D< complex< long double > > & a,
            tensor2D< int > & finalsubspace,
            int * subdim,
            int & numspace )
```

Routine to identify symmetry subspace by the analysis of the interaction matrix.

Parameters

in	*maxl*	size of the interaction matrix a
in	*a*	matrix of type tensor2D<complex <long double>=""> >containing the constant (structure costant) of the various verma module and perturbation, of which we want to identify subspace
out	*finalsubspace*	tensor2D< int> that will contain in the row the indeces of the Verma module belonging the each subspace
out	*subdim*	Array that contains the number of Verma module in each subspace
out	*numspace*	Number of subspace found

4.2 characters.h File Reference

Set of helper functions to calculate integers partitions and manage GiNaC computation of character expansion.

```
#include <ginac/ginac.h>
#include "tensors.h"
```

Functions

- ex invphi (int k, symbol x)

  Calculates the inverse denominator of the characters formula up to order k.
- ex ksymb1 (int r, int s, int p, int q, int k, symbol x)

  Calculates the first contribution of the numerator of the characters formula up to order k.
- ex ksymb2 (int r, int s, int p, int q, int k, symbol x)

  Calculates the second contribution of the numerator of the characters formula up to order k.
- ex chisymb (int r, int s, int p, int q, int l, symbol x)

  Calculates the symbolic expression of the characters formula for given Verma module and model, up to order k.
- void characexpan (int r, int s, double p, double q, int l, int *leveldim)

  Calculates the characters expantion, to get the number of independent states at each level for a given Verma module.
- int partitions (int)

  Calculates the number of partions of the given integer.
- int determine_partition (int n, tensor2D< int > &results)

  Calculates the partions of integers up to the given integer.
- void part (int, int *, int, tensor2D< int > &, int &)

  Helper function, recursive component of determine_partition.

4.2.1 Detailed Description

Set of helper functions to calculate integers partitions and manage GiNaC computation of character expansion.

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

Copyright

GNU Public License.

4.2.2 Function Documentation

4.2.2.1 characexpan()

```
void characexpan (
            int r,
            int s,
            double p,
            double q,
            int l,
            int * leveldim )
```

Calculates the characters expantion, to get the number of independent states at each level for a given Verma module.

Parameters

in	*r*	The first index defining the Verma Module
in	*s*	The second index defining the Verma Module
in	*p*	The first index defining the model M_(p,q)
in	*q*	The second index defining the model M_(p,q)
in	*l*	Maximum level to perform the expansion
out	*leveldim*	Array of lenght l+1, contains the number of indenpendent states at each level

4.2.2.2 chisymb()

```
ex chisymb (
            int r,
            int s,
            int p,
            int q,
            int l,
            symbol x )
```

Calculates the symbolic expression of the characters formula for given Verma module and model, up to order k.

Parameters

in	*r*	The first index defining the Verma Module
in	*s*	The second index defining the Verma Module
in	*p*	The first index defining the model M_(p,q)
in	*q*	The second index defining the model M_(p,q)
in	*k*	maximum exponent included in the product in the denominator
in	*x*	GiNac symbol object, contains the variable used for the symbolic calculation return GiNaC expression containing the characters for the given Verma module and model up to order k

4.2.2.3 determine_partition()

```
int determine_partition (
            int n,
            tensor2D< int > & results )
```

Calculates the partions of integers up to the given integer.

Parameters

in	*n*	integer to calculate the partions
out	*results*	Reference to tensor2D<int> that will contain the different partions of the given integer, one per row return The number of partions for the given integer

Calculates the partions of integers up to the given integer.

Kept for reference

4.2.2.4 invphi()

```
ex invphi (
            int k,
            symbol x )
```

Calculates the inverse denominator of the characters formula up to order k.

Parameters

in	*k*	maximum exponent included in the product in the denominator
in	*x*	GiNac symbol object, contains the variable used for the symbolic calculation return GiNaC expression containing the symbolic inverse denominator up to order k

Calculates the inverse denominator of the characters formula up to order k.

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

Copyright

GNU Public License.

4.2.2.5 ksymb1()

```
ex ksymb1 (
            int r,
            int s,
            int p,
            int q,
            int k,
            symbol x )
```

Calculates the first contribution of the numerator of the characters formula up to order k.

Parameters

in	*r*	The first index defining the Verma Module
in	*s*	The second index defining the Verma Module
in	*p*	The first index defining the model M_(p,q)
in	*q*	The second index defining the model M_(p,q)
in	*k*	maximum exponent included in the product in the denominator
in	*x*	GiNac symbol object, contains the variable used for the symbolic calculation return GiNaC expression containing the symbolic first contribution to the numerator up to order k

4.2.2.6 ksymb2()

```
ex ksymb2 (
            int r,
            int s,
            int p,
            int q,
            int k,
            symbol x )
```

Calculates the second contribution of the numerator of the characters formula up to order k.

Parameters

in	*r*	The first index defining the Verma Module
in	*s*	The second index defining the Verma Module
in	*p*	The first index defining the model M_(p,q)
in	*q*	The second index defining the model M_(p,q)
in	*k*	maximum exponent included in the product in the denominator
in	*x*	GiNac symbol object, contains the variable used for the symbolic calculation return GiNaC expression containing the symbolic second contribution to the numerator up to order k

4.2.2.7 partitions()

```
int partitions (
```

```
            int  )
```

Calculates the number of partions of the given integer.

Parameters

in	*k*	The integer to calculate the partion return Number of partions of the given integer

4.3 chirME.h File Reference

Class that contains the chiral matrix elements of perturbing operators and operator to calculate expectation values.

```
#include "parameters.h"
#include "states.h"
#include "tensors.h"
#include "filenames.h"
```

Classes

- class chirME

  Class that contains the chiral matrix elements of perturbing operators and of the operator to calculate expectation values.

4.3.1 Detailed Description

Class that contains the chiral matrix elements of perturbing operators and operator to calculate expectation values.

authors: Giuseppe Piero Brandino - eXact-lab s.r.l., Robert Konik - BNL last revision: December 2016

4.4 config.h File Reference

Configuration file holding version number, some hard-coded limits that effect array dimensions, constants and macros.

Macros

- #define **VERSION** "1.0.0"
- #define MaxPart 1500

  Maximum number of partitions of integers (so the maxium possible number of different strings of lowering opertaros) in a chiral expectation value.
- #define MaxNumGrade 50

  Maximum number of operators (both field and lowering operator) in a chiral expectation value.
- #define **SIGN**(a, b) ((b) >= 0.0 ? fabsl(a) : -fabsl(a))
- #define **SIGNd**(a, b) ((b) >= 0.0 ? fabs(a) : -fabs(a))
- #define **pi** 3.1415926535897

4.4.1 Detailed Description

Configuration file holding version number, some hard-coded limits that effect array dimensions, constants and macros.

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

Copyright

GNU Public License.

4.4.2 Macro Definition Documentation

4.4.2.1 MaxNumGrade

```
#define MaxNumGrade 50
```

Maximum number of operators (both field and lowering operator) in a chiral expectation value.

It could be in principle be calculated, but for any practical purposes given value is enough. Increasing this will typically require to increase stack size.

4.4.2.2 MaxPart

```
#define MaxPart 1500
```

Maximum number of partitions of integers (so the maxium possible number of different strings of lowering opertaros) in a chiral expectation value.

It could be in principle be calculated, but for any practical purposes given value is enough. Increasing this will typically require to increase stack size.

4.5 cpt.h File Reference

Set of helper function to calculate structure constant of Minimal CFTs.

```
#include <complex>
```

Functions

- long double hrs (long double r, long double s, long double p, long double q)

  Component of the structure constant formula, used by std::complex<long double> dot.
- long double dotmu (int ll, int l, long double ro, long double ro1)

  Component of the structure constant formula, used by std::complex<long double> dot.
- long double dotc (int ll, int l, long double ro, long double ro1, long double kn1, long double kn, long double ks1, long double ks, long double kp1, long double kp)

  Component of the structure constant formula, used by std::complex<long double> dot.
- long double dota (int kn1, int kn, long double ro, long double ro1)

  Component of the structure constant formula, used by std::complex<long double> dot.
- std::complex< long double > dot (int p, int q, int s1, int r1, int s2, int r2, int s3, int r3)

  Routine to calculate the structure constant of a Minimal CFT M_(p,q) for a given triad of fields phi_(r1,s1) phi_(r2,s2) phi(r3,s3)

4.5.1 Detailed Description

Set of helper function to calculate structure constant of Minimal CFTs.

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

Copyright

GNU Public License.

4.5.2 Function Documentation

4.5.2.1 dot()

```
std::complex<long double> dot (
            int p,
            int q,
            int s1,
            int r1,
            int s2,
            int r2,
            int s3,
            int r3 )
```

Routine to calculate the structure constant of a Minimal CFT M_(p,q) for a given triad of fields phi_(r1,s1) phi_(r2,s2) phi(r3,s3)

Parameters

in	*p*	first index of the minimal model
in	*q*	second index of the minimal model
in	*r1*	first Kac index of the first field
in	*s1*	second Kac index of the first field
in	*r2*	first Kac index of the second field
in	*s2*	second Kac index of the second field
in	*r3*	first Kac index of the third field
in	*r3*	second Kac index of the third field return std::complex<long double> containg the structure constant for the given fields.

Beware that in the function call the s index (second index) comes before of the r index (first index) for every field

4.6 diag.h File Reference

Contains the declaration of the diag Class, which provides a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double.

```
#include "tensors.h"
```

Classes

- class diag

  Class containg a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double.

4.6.1 Detailed Description

Contains the declaration of the diag Class, which provides a set of c-programs that diagonalize matrices based on code found in Numerical Recipes, for both double and long double.

Author

Robert Konik - BNL, Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

November 2016

4.7 element_computation.h File Reference

Routines to calculate matrix element and overlap of conformal states.

```
#include "config.h"
```

Functions

- long double fac (int n)

  Return the factorial of the given integer.
- long double element_computation (long double C, long double grade[MaxNumGrade], int n, int m, long double hr, long double hl, long double coeff, long double Del)

  Calculates the matrix element of a primary field phi_(hr,hs) on the conformal basis.

4.7.1 Detailed Description

Routines to calculate matrix element and overlap of conformal states.

Author

Robert Konik - BNL, Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

November 2016

4.7.2 Function Documentation

4.7.2.1 element_computation()

```
long double element_computation (
              long double C,
              long double grade[MaxNumGrade],
              int n,
              int m,
              long double hr,
              long double hl,
              long double coeff,
              long double Del )
```

Calculates the matrix element of a primary field phi_(hr,hs) on the conformal basis.

Parameters

in	*C*	The central charge of the model
in	*grade*	Array of size MaxNumGrade (defined in config.h). Contains the sequence of indeces of lowering and rising operator that define the bra and ket states, and 0 for the primary field, read from left to right
in	*n*	Length of the string or rising and lowering operators, including the primary field
in	*m*	Position of the primary field in the string
in	*hr*	Weight of the primary field from which the current conformal ket states is descending
in	*hl*	Weight of the primary field from which the current conformal bra states is descending
in	*coeff*	Coefficient to multiply the matrix element. Used internally in recursive calls
in	*Del*	The conformal weight of the field whose matrix element are computed. 0 indicates identity, so the overlaps of the conformal basis states are computed. return The matrix element

4.7.2.2 fac()

```
long double fac (
            int n )
```

Return the factorial of the given integer.

Parameters

in	*n*	The integer to calculate the factorial return The factorial of n

4.8 gamma.h File Reference

Computation of gamma function.

Functions

- double gamma (double x)

  Computation of gamma function.

4.8.1 Detailed Description

Computation of gamma function.

Algorithms and coefficient values from "Computation of Special Functions", Zhang and Jin, John Wiley and Sons, 1996.

Authors

C. Bond. All rights reserved.

Date

2003

4.8.2 Function Documentation

4.8.2.1 gamma()

```
double gamma (
            double x )
```

Computation of gamma function.

[in] x Value to calculate gamma(x) return gamma function of argument 'x'.

NOTE: Returns 1e308 if argument is a negative integer or 0, or if argument exceeds 171.

4.9 parameters.h File Reference

Input parameter class.

```
#include "tensors.h"
#include <complex>
```

Classes

- class parameters

  Input parameter class.

4.9.1 Detailed Description

Input parameter class.

Big fuzzy object that contains

- input paramenter
- Kac table
- structure constant
- verma module in each subspace

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

November 2017

4.10 states.h File Reference

Class that contains and calculates chiral states by removing null vectors from each verma module.

```
#include "parameters.h"
#include "filenames.h"
```

Classes

- class states

  Class that contains and calculates chiral states by removing null vectors from each verma module.

4.10.1 Detailed Description

Class that contains and calculates chiral states by removing null vectors from each verma module.

Author

Giuseppe Piero Brandino - eXact-lab s.r.l., Robert Konik - BNL

Date

November 2017

4.11 tensors.h File Reference

Set of helper templates class to manage 2D, 3D, 4D, 5D, 6D tensors.

```
#include <iostream>
#include <cstdlib>
```

Classes

- class tensor2D< T >

  Template class for 2D tensors This classes do not provide an allocator.
- class tensor3D< T >

  Template class for 3D tensors This classes do not provide an allocator.
- class tensor4D< T >

  Template class for 4D tensors This classes do not provide an allocator.
- class tensor5D< T >

  Template class for 5D tensors This classes do not provide an allocator.
- class tensor6D< T >

  Template class for 6D tensors This classes do not provide an allocator.

4.11.1 Detailed Description

Set of helper templates class to manage 2D, 3D, 4D, 5D, 6D tensors.

This classes do not provide an allocator. They are allocated automatically using the costum constructor tesor?D(int int ....) The default constructor can be used, but only to create dummy tensors, that needs to be assigned using the assignment operator (=)

Memebers are all public.

- n1, n2, ... contains the dimensions
- the pointer data contains the data in col major order, such that it can be pass to BLAS/LAPACK easily as A.data
- reading and writing single elements is done using the () operator so element 1,2,5 of tensor A is accessed A(1,2,5)

Author

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

4.12 truspace.cpp File Reference

Main program of Truspace (driver routine).

```
#include <iostream>
#include "parameters.h"
#include "worker.h"
```

Functions

- int **main** (int argc, char **argv)

4.12.1 Detailed Description

Main program of Truspace (driver routine).

Acts a a driver routine, reading input files and calling worker class to perform TCSA if required

Authors

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

October 2017

Copyright

GNU Public License.

4.13 worker.h File Reference

Contains driver Class for states, matrix element and NRG calculation.

```
#include "parameters.h"
```

Classes

- class worker

  Driver Class for states, matrix element and NRG calculation.

4.13.1 Detailed Description

Contains driver Class for states, matrix element and NRG calculation.

It takes care of

- looping through the subspace found by parameters::analyze_subspace
- finding chiral and non-chiral base states in each subspace
- calling NRG routine on each subspace
- calculating Matrix element on the eigenstates found, in each subspace. If the operator connects a set of subspaces, then those subspaces are diagonalized at the same time

Author

Giuseppe Piero Brandino - eXact-lab s.r.l.

Date

November 2017

Index

allMomSubspace
    parameters, 17

block.h, 41
    blockrec, 41
blockrec
    block.h, 41

C
    parameters, 17
calculate_ExpVal
    chirME, 6
calculate_chirME_for_ExpVal
    chirME, 6
calculate_chiral
    states, 21
calculate_nonchiral
    states, 22
calculate_or_read_chirME
    chirME, 7
characexpan
    characters.h, 44
characters.h, 43
    characexpan, 44
    chisymb, 44
    determine_partition, 45
    invphi, 45
    ksymb1, 45
    ksymb2, 46
    partitions, 46
chirME.h, 47
chirME, 5
    calculate_ExpVal, 6
    calculate_chirME_for_ExpVal, 6
    calculate_or_read_chirME, 7
    chirME, 6
    me_chi_ortho, 7
    me_chi_ortho_op_l, 7
    me_chi_ortho_op_r, 8
chisymb
    characters.h, 44
config.h, 47
    MaxNumGrade, 48
    MaxPart, 48
cpt
    parameters, 17
cpt.h, 48
    dot, 49
cptsmall
    parameters, 17
cstate_lev
    states, 23
cstate_mode
    states, 23
cstate_nmode
    states, 23
cstate_ortho_lev
    states, 23
cstate_ortho_mode
    states, 23
cstate_ortho_nmode
    states, 23

determine_partition
    characters.h, 45
diag, 8
    shell_double, 9
    shell_ld, 9
    tqli_double, 10
    tqli_ld, 10
    tred2_double, 11
    tred2_ld, 11
diag.h, 50
dot
    cpt.h, 49

element_computation
    element_computation.h, 51
element_computation.h, 50
    element_computation, 51
    fac, 52

fac
    element_computation.h, 52
filenames, 12
finalsubspace
    parameters, 18

gamma
    gamma.h, 52
gamma.h, 52
    gamma, 52

Hamiltonian, 12

intMatrix
    parameters, 18
invphi
    characters.h, 45

ksymb1

characters.h, 45
ksymb2
characters.h, 46

MaxChiralStates
states, 24
MaxChiralVerma
states, 24
maxEigen
parameters, 18
MaxNumGrade
config.h, 48
MaxPart
config.h, 48
MaxStates
states, 24
maxp
parameters, 18
me_chi_ortho
chirME, 7
me_chi_ortho_op_l
chirME, 7
me_chi_ortho_op_r
chirME, 8

NRG, 13
run, 13
nSweep
parameters, 19
Nsize
parameters, 18
nstates_lev
states, 24
nstates_lev_each
states, 24
Nstep
parameters, 18

operator()
tensor2D, 27
tensor3D, 29, 30
tensor4D, 32
tensor5D, 35
tensor6D, 38, 39
operator=
tensor2D, 27
tensor3D, 30
tensor4D, 33
tensor5D, 36
tensor6D, 39

parameters, 14
allMomSubspace, 17
C, 17
cpt, 17
cptsmall, 17
finalsubspace, 18
intMatrix, 18
maxEigen, 18
maxp, 18
nSweep, 19
Nsize, 18
Nstep, 18
parameters, 16
print, 17
subdim, 19
vermadim, 19
vermadimcomul, 19
parameters.h, 53
partitions
characters.h, 46
print
parameters, 17

read_chiral
states, 22
run
NRG, 13

shell_double
diag, 9
shell_ld
diag, 9
state_left_lev
states, 24
state_nonchiral_lev
states, 25
state_right_lev
states, 25
states, 19
calculate_chiral, 21
calculate_nonchiral, 22
cstate_lev, 23
cstate_mode, 23
cstate_nmode, 23
cstate_ortho_lev, 23
cstate_ortho_mode, 23
cstate_ortho_nmode, 23
MaxChiralStates, 24
MaxChiralVerma, 24
MaxStates, 24
nstates_lev, 24
nstates_lev_each, 24
read_chiral, 22
state_left_lev, 24
state_nonchiral_lev, 25
state_right_lev, 25
states, 21
states.h, 54
subdim
parameters, 19

tensor2D< T >, 25
tensor2D
operator(), 27
operator=, 27
tensor2D, 26
tensor3D< T >, 28

tensor3D
    operator(), 29, 30
    operator=, 30
    tensor3D, 29
tensor4D< T >, 31
tensor4D
    operator(), 32
    operator=, 33
    tensor4D, 32
tensor5D< T >, 33
tensor5D
    operator(), 35
    operator=, 36
    tensor5D, 34
tensor6D< T >, 36
tensor6D
    operator(), 38, 39
    operator=, 39
    tensor6D, 38
tensors.h, 54
tqli_double
    diag, 10
tqli_ld
    diag, 10
tred2_double
    diag, 11
tred2_ld
    diag, 11
truspace.cpp, 55

vermadim
    parameters, 19
vermadimcomul
    parameters, 19

worker, 40
worker.h, 56